

Lab 3: Architecture Justification

Problem Statement #10 – Student Club Event Ticketing & Budget Portal

Architecture Selection: We chose Microservices Architecture for the Student Club Event Ticketing & Budget Portal.

Architectural Choice

The portal is split into independently deployable services behind an API Gateway: **Approval Service, Ticket Service, Validation Service** and **Notification Service**, with dedicated data stores for approval and ticketing. Clients (Web Portal UI, QR Scanner App) talk only to the gateway. Layered architecture was rejected because one deployable unit must scale as a whole and couples budget workflow to gate check-in. Client-Server was rejected because a single central server is a single point of failure and a bottleneck when many attendees are scanned at once.

Two Reasons for This Choice

1. Very different load profiles (NFR-001). Budget approvals (FR-001) are a handful of low-volume transactions, whereas QR validation (FR-005) arrives in sharp bursts at venue entrances and must complete in under 100 ms. With microservices the Validation Service is scaled out on its own during events, without paying to scale the approval workflow.

2. Independent business domains and fault isolation. The three-stage approval chain (Faculty Coordinator → Finance Office → Dean) and QR ticketing are separate domains that change for different reasons. If the Notification or Approval Service is down, attendees with already-issued tickets can still be validated at the gate. The Ticket Service calls `IEventStatus` so tickets are issued only for events marked Budget Approved.

Security Advantage

The API Gateway is the single entry point: it authenticates users (JWT) and enforces **role-based access control** so only the Faculty Coordinator, Finance Officer or Dean can approve at their own stage and only Campus Admin can manage events (NFR-002). Internal services and databases are not exposed to the internet. Keeping internal services and databases behind the API Gateway limits their exposure and reduces the impact of a compromise. QR codes are generated only by the Ticket Service and checked server-side, which prevents forged or reused tickets (FR-004, FR-005).

Performance Benefit

The Validation Service is stateless and horizontally scalable, and reads a dedicated, indexed Ticket DB by unique QR code. It is never slowed by approval or notification traffic, so the 100 ms check-in target can be met under peak load. Notifications are sent asynchronously through a message queue (`INotify`), so approving or rejecting a proposal never waits on e-mail delivery.

Trade-off Acknowledged

Microservices add operational complexity (more deployments, network calls, and eventual consistency between services). For a university portal with a burst-heavy check-in path and strict role separation, this cost is justified by better scalability, isolation and security.
